Internet Programming

CSE341
Lab 4

Advanced JavaScript Features

ES6 Features – Shortcuts for Optimizing Code

- ES6 (short for ECMAScript 6) is a major update to the JavaScript language, released in 2015.
- Before ES6:
  - Code was verbose (too many lines for simple tasks).
  - There were no proper ways to write classes, modules, or constants.
  - Asynchronous code (like waiting for network data) was hard to manage.

- ES6 introduced many new features that made JavaScript cleaner, more consistent, and developer-friendly. Introduced modern syntax like:
  - let/const (We saw in previous lab)
  - arrow functions
  - template literals
  - destructuring, (New concepts to discuss today)
  - spread/rest
  - Classes
  - Modules import and export

Recap Arrays and their Methods

- Definition:
  - An array is an ordered collection of values (elements) accessible by numeric indices starting at 0.

- Characteristics (special for JavaScript):
  - Can contain mixed types (numbers, strings, objects, functions).
  - Length is dynamic and adjusts as elements are added/removed (arr.length).

- Mutating Method(change original array): push, pop, shift, unshift, splice, sort, reverse.
  - Ex: JS Snippet

```
const mixed = [1, 'two', {name:'A'}, [3,4]];
mixed.push("mango"); // [1, 'two', {name:'A'}, [3,4], "mango"]
```

- Non-mutating Method(return new array/value): map, filter, reduce, slice, concat, flatMap.
  - Ex: JS Snippet

```
const nums = [1,2,3];
const doubled = nums.map(n => n * 2); // [2,4,6]
```

Finding Elements in Arrays (Return one value)

Method	Description	Example
`indexOf()`	Find index of value	`fruits.indexOf("banana");`
`includes()`	Check existence	`fruits.includes("apple");`
`find()`	Find first element that matches	`[3,7,10].find(n => n > 5); // 7`
`findIndex()`	Get index of first match	`[3,7,10].findIndex(n => n > 5); // 1`

Transforming Arrays

Method	Description	Example
`map()`	Create new array by applying a function	`[1,2,3].map(n => n * 2); // [2,4,6]`
`filter()`	Return array elements that match condition	`[1,2,3,4].filter(n => n % 2 === 0); // [2,4]`
`reduce()`	Combine all elements into single value	`[1,2,3,4].reduce((sum,n) => sum+n, 0); // 10`

Example Code

```html
<!DOCTYPE html>
<html>
<body>
  <h3>Filter Even Numbers</h3>
  <input id="nums" placeholder="Enter numbers: 1,2,3,4">
  <button onclick="filterEven()">Filter</button>
  <p id="result"></p>

  <script>
    const filterEven = () => {
      const input = document.getElementById('nums').value;
      const arr = input.split(',').map(Number);
      const even = arr.filter(n => n % 2 === 0);
      document.getElementById('result').innerText = `Even numbers: ${even}`;
    }
  </script>
```

Filter Even Numbers

Enter numbers: 1,2,3,4 | Filter

Filter Even Numbers

4, 67, 58, 321, 3468 | Filter

Even numbers: 4,58,3468

Chaining Array Methods: Predict the Code Output

- Chaining lets you express multi-step data pipelines concisely. Example:

```
const numbers = [1, 2, 3, 4, 5];

const evenSquares = numbers
    .filter(n => n % 2 === 0)
    .map(n => n ** 2)
    .reduce((sum, n) => sum + n, 0);

console.log(evenSquares);
```

- You can execute multiple array methods in sequence. Make sure to place the first operation first followed by others:
  Main_array.1st_method.2nd_method. Etc…

Example Code

```
<h3>Search For Elements in a list</h3>
<input id='q' placeholder='search' />
<ul id='list'></ul>
<script>
  const items = ['apple','banana','cherry','avocado'];

  const render = list => {
  document.getElementById('list').innerHTML =
      list.map(i => `<li>${i}</li>`).join('');
  };

  document.getElementById('q').addEventListener('input', e => {
  const q = e.target.value.toLowerCase();
  render(items.filter(x => x.includes(q)));
  });

  render(items);
</script>
```

Search For Elements in a list

search

- apple
- banana
- cherry
- avocado

Search For Elements in a list

ch

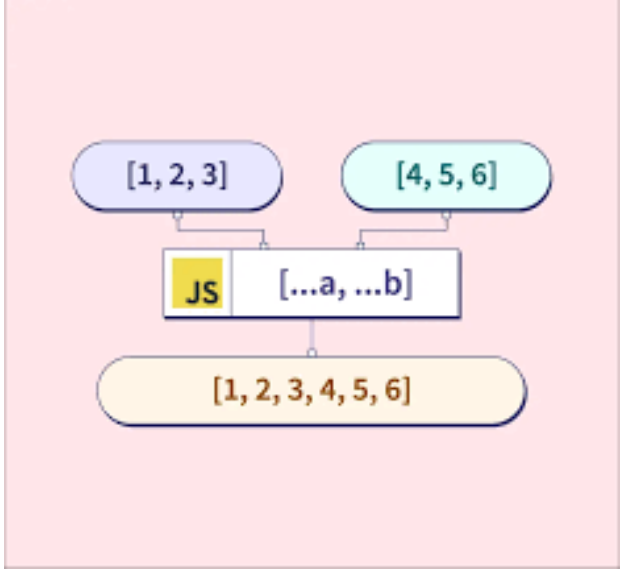
- cherry

Spread and Rest Operator (…)

- ES6 provides “Spread Operator” to expand elements of an array
- Ex:

```
const a = [0, 1, 2];
const merged = [...a, 3, 4, 5];   //[0, 1, 2, 3, 4, 5]
```

- Common use cases for the spread operator:
  - Copying arrays: `const copy = [...arr];`
  - Merging: `const merged = [...a, ...b];`
  - Spreading into function calls: `Math.max(...nums);`



- Rest Operator collects multiple arguments into an array.
- Commonly used in function args:

```
function sum(...numbers) {
  return numbers.reduce((a, b) => a + b);
}
console.log(sum(1,2,3)); // 6
```

- Note: spread/rest use same token (...) but context decides meaning.

Array Destructuring

- ES6 allows for extracting data from arrays based on the context of the new variables used to store the extracted data.
- Examples:

```
const [first, second] = ["apple", "banana", "cherry"];
console.log(first); // apple

const [head, ...tail] = [1,2,3,4];
console.log(tail); // [2,3,4]
```

Array Utilities

- `Array.from(iterable)`: convert iterables or array-like objects (NodeList, arguments) to arrays.

```
const str = "hello";
const letters = Array.from(str); // ['h','e','l','l','o’]
```

- `Array.of(...)`: create array from arguments safely.

```
const nodes = Array.from(document.querySelectorAll('li')).map(n => n.textContent);
```

Example Code

```
<script>
  // ======== 1. SPREAD EXAMPLE ========
  const existingUsers = ['Alice', 'Bob', 'Charlie'];
  const newUsers = ['Diana', 'Evan'];

  document.getElementById('mergeBtn').addEventListener('click', () => {
    const merged = [...existingUsers, ...newUsers];
    document.getElementById('spreadOutput').innerHTML =
      `<strong>Merged Users:</strong> ${merged.join(', ')}`;
  });


  // ======== 2. REST EXAMPLE ========
  function calculateTotal(...prices) {
    return prices.reduce((total, price) => total + price, 0);
  }

  document.getElementById('cartBtn').addEventListener('click', () => {
    const total = calculateTotal(12.99, 8.50, 4.25, 6.99);
    document.getElementById('restOutput').innerHTML =
      `Items: $12.99, $8.50, $4.25, $6.99 <br><strong>Total:</strong> $${total.toFixed(2)}`;
  });


  // ======== 3. DESTRUCTURING EXAMPLE ========
  const apiResponse = ['Mina', 'mina@example.com', 'Admin'];

  document.getElementById('apiBtn').addEventListener('click', () => {
    const [username, email] = apiResponse;
    document.getElementById('destructOutput').innerHTML =
      `User: <strong>${username}</strong> <br>Email: ${email}`;
  });
</script>
```

Merge User Lists

Merged Users: Alice, Bob, Charlie, Diana, Evan

Calculate Total

Items: $12.99, $8.50, $4.25, $6.99

Total: $32.73

Show API User Info

User: **Mina**

Email: mina@example.com

Sets and Maps

Sets

- A Set is a collection of unique values (no duplicates allowed).
- It’s useful when you want to remove duplicates from an array or check membership quickly.
- Set maintains insertion order.
- Methods include:
  - add(value) → adds a value.
  - has(value) → checks existence.
  - delete(value) → removes a value.
  - size → number of items.
  - forEach(callback) → iterate elements.

```
const names = ['Ali', 'Mona', 'Ali', 'Sara'];
const unique = new Set(names);
document.getElementById('output').textContent
= [...unique].join(', ‘);
 // "Ali, Mona, Sara“  (Notice why
“...unique” is used not “unique”)
```

Maps

- A Map stores key-value pairs (like objects but more flexible).
- Any type can be a key not only strings.
- It remembers insertion order and is great for dynamic key-value storage.
- Methods include:
  - set(key, value) → adds/updates a pair.
  - get(key) → retrieves value.
  - has(key) → checks if key exists.
  - delete(key) / clear().
  - forEach(callback) → iterate entries.

```
const users = new Map([["Ali", 20], ["Mona", 25]]);
users.forEach(
(age, name) => console.log(`${name} is ${age}`));
```

Example Code

```
  <h2>User Management Example</h2>
<button id="add">Add User</button>
<ul id="nameList"></ul>

<script type="module">
  const users = new Map();
  const ids = new Set();

  document.getElementById('add').addEventListener('click', () => {
    const name = prompt('Enter user name:');
    const age = prompt('Enter age:');
    if (!name || ids.has(name)) return alert('User exists or invalid');

    users.set(name, age);
    ids.add(name);
    render();
  });

  function render() {
    const list = document.getElementById('nameList');
    list.innerHTML = [...users].map(([n, a]) => `<li>${n} (${a})</li>`).join('');
  }
</script>
```

User Management Example

Add User

- Hamdy (26)

Classes in JavaScript

- A class is a blueprint for creating objects.
- Introduced in ES6 for cleaner, more structured object-oriented code.
- Internally built on prototypes, but easier to read and maintain.

- Example:

```
class Person {
  constructor(name, age) { this.name = name;
this.age = age; }

greet() { console.log(`Hi, I'm ${this.name}`);
}
}

const ali = new Person('Ali', 25);
ali.greet();
```

The constructor() Method

- Runs automatically when a new object is created.
- Used to initialize object properties.
- Example:

```
class Car {
  constructor(model, year) {
    this.model = model;
    this.year = year;
  }
}
const car1 = new Car('Toyota', 2023);
```

Instance Methods

- Methods defined inside a class but outside the constructor.
- Each object can access them independently.
- Example:

```
class Calculator {
  add(a, b) { return a + b; }
  subtract(a, b) { return a - b; }
}
const calc = new Calculator();
console.log(calc.add(5,3));
```

Class Fields

- Introduced in ES2022 for direct property declaration.
- No need to use constructor() for initializing fields.

- Example:

```
class Counter {
  count = 0;
  increment() { this.count++; }
}
const c = new Counter();
c.increment();
console.log(c.count); // 1
```

Static Methods and Properties

- Declared with the 'static' keyword.

- Belong to the class itself, not to instances.

- Example:

```
class MathHelper {
  static square(x) { return x * x; }
}
console.log(MathHelper.square(5)); // 25
```

Inheritance: extends & super

- Enables one class to inherit properties and methods from another.
- 'extends' creates the child class, and 'super()' calls the parent constructor.
- Example:

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { console.log(`${this.name}
makes a noise.`); }
}
class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
  speak() { console.log(`${this.name}
barks!`); }
}
```

Getters and Setters

- Control access to object data with computed or validated properties.
- Example:

```
class Temperature {
  constructor(celsius) { this.celsius =
celsius; }
  get fahrenheit() { return this.celsius
* 1.8 + 32; }
  set fahrenheit(value) { this.celsius =
(value - 32) / 1.8; }
}
const temp = new Temperature(0);
console.log(temp.fahrenheit); // 32
```

Common Pitfalls and Best Practices with Classes in JS

- Always use 'new' when creating class instances.
- Arrow functions don’t bind their own 'this' context use normal methods.
- Class methods are not hoisted must be defined before used.
- Keep static methods for utility purposes only.
- Classes in JS are different than classes in CSS.

Example Code

```
  <h2>JavaScript Classes Examples</h2>

<input id='desc' placeholder='Task description'>
<button id='addBtn'>Add Task</button>
<ul id='list2'></ul>
<script>
  class Task {
    constructor(description) {
      this.description = description;
      this.completed = false;
    }
    toggle() { this.completed = !this.completed; }
  }
  const tasks = [];
  document.getElementById('addBtn').onclick = () => {
    const text = document.getElementById('desc').value;
    const t = new Task(text);
    tasks.push(t);
    document.getElementById('list2').innerHTML += `<li>${t.description}</li>`;
  }
</script>
```

JavaScript Classes Examples

Finish Lab Task | Add Task

- Finish Lab Task

Function Default Parameters

- Used to provide function arguments default values to be used if the user did not provide any arguments.

```
function greet(name = "Guest") {
  console.log(`Hello, ${name}!`);
}
greet(); // Hello, Guest!
```

Function Modules Importing and Exporting

- Help in splitting JS functions into different files based on modules for easier maintenance.
- Use “import” to use functions from other files (functions should have type “export” to be seen out of its own file)

```
// In file Math.js
export const add = (a, b) => a + b;

//In file main.js
import { add } from './math.js';
console.log(add(2, 3)); // 5
```

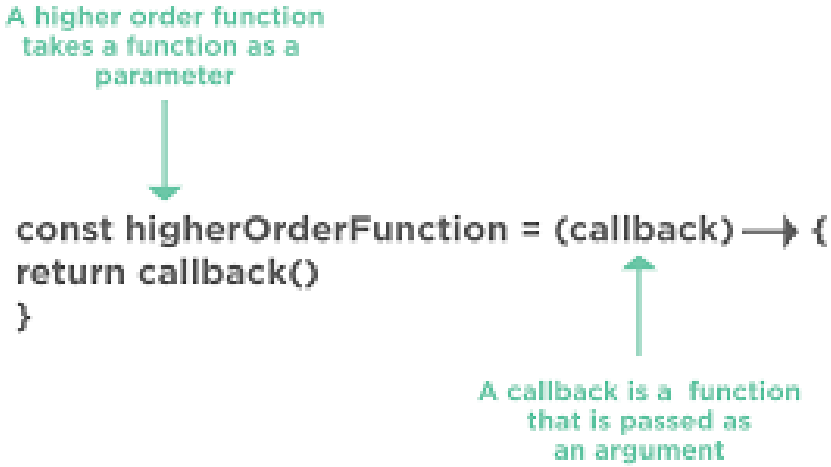
- ES6 introduced modules for code organization. Each JavaScript file can be treated as a separate module.
- export exposes variables, functions, or classes. Use import that brings them into another file.
- You must use <script type="module"> in HTML for this to work in browsers.

```
<script type="module" src="main.js"></script>
```

- Browsers treat running JS locally without a webserver as origin null, and block features such as module imports or fetch() requests for security reasons.

Function Callbacks

- A callback is a function passed to another function and invoked later. It allows one function to call another when an operation completes.
- Used for events, timers, and older async APIs.
- Pitfalls: callback pyramid (callback hell), inversion of control, error handling complexity.
- Synchronous callbacks
  - The callback executes immediately after being called.
  - JavaScript waits for it to finish before moving on.
- Asynchronous callbacks
  - In asynchronous callbacks, JavaScript doesn't wait for the function to finish and continues executing other code.
  - The callback runs later after a delay or external event.
  - Common in network requests, file reading, or timers.

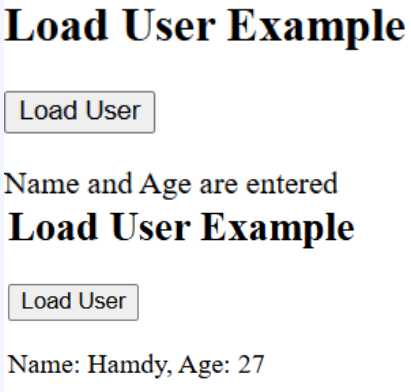


Example Code

```
<h2>Load User Example</h2>
<button id='loadBtn'>Load User</button>
<p id='output'></p>
<script>
  function fetchUser(sync_callback, async_callback) {
    const name = prompt('Enter your name:');
    sync_callback(name);

    const age = prompt('Enter your age:');

    setTimeout(() => {
     const user = { name: `${name}`, age: `${age}` };
     async_callback(user);
    }, 2000);

    document.getElementById('output').textContent = 'Name and Age are entered';
  }
  function greetUser(user) {
    alert(`Hello, ${user}!`);

  }

  function displayUser(user) {
    document.getElementById('output').textContent = `Name: ${user.name}, Age: ${user.age}`;

  }
  document.getElementById('loadBtn').addEventListener('click', () => fetchUser(greetUser,displayUser));
</script>
```

Sync callback. The code will run callback and finish it first to continue other codes

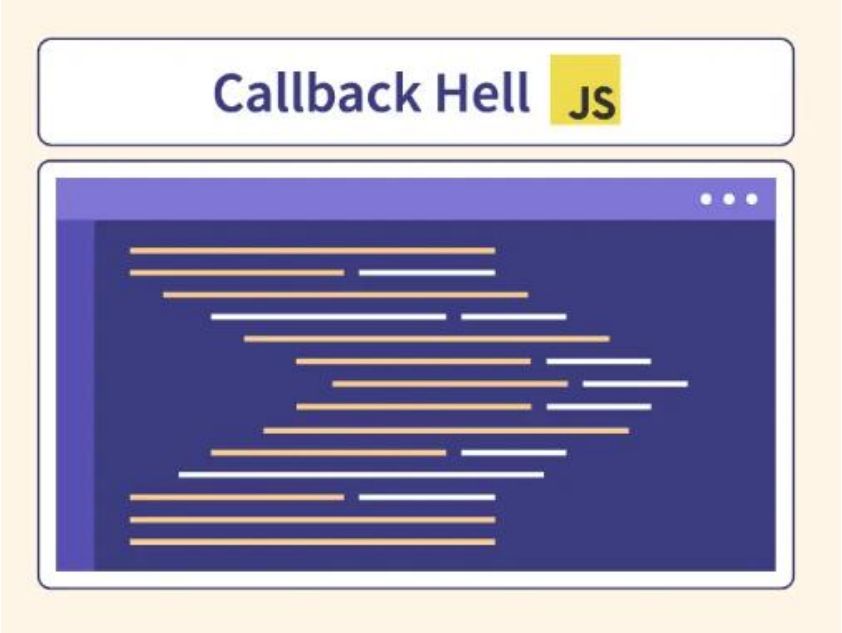
Async callback called through setTimeout. Code inside will run after 2 seconds without blocking other codes



Callback Hell

- When multiple asynchronous callbacks are nested, code becomes messy:

```
getData(() => {
  processData(() => {
    saveData(() => {
      displayData(() => {
        console.log('Done!');
      });
    });
  });
});
// Difficult to debug and maintain.
```



Lab Task

My Interactive To Do List – No AI Allowed

- Build a simple To-Do List web app using HTML, CSS, and JavaScript.
- Specifications:
- **HTML structure (minimum requirements)**
  - An input field for task description.
  - An “Add Task” button.
  - A list (<ul> or) to display tasks.
- **JavaScript (minimum requirements)**
  - Create a Task class
  - Store tasks in an array.
  - Add tasks dynamically to the page using innerHTML or DOM manipulation.
  - Allow toggling a task’s completion state by clicking on it (change text style or color).
  - Allow deleting a task.
  - Use simple CSS to mark completed tasks with a strike-through or color change.

- Make it look good (experiment with CSS)

My Interactive “Pro” To Do List – For Vibe Coders

- **Additional requirements:**
- **Persistence:**
  - Save and load tasks using localStorage.
- **Filtering & Sorting:**
  - Add buttons to filter tasks: *All*, *Completed*, *Incomplete*.
  - Allow sorting alphabetically or by creation time.
- **Enhancements:**
  - Add a timestamp when a task is created.
  - Add an “Edit” feature to rename a task.
- **UI Upgrade:**
  - Use dynamic icons (✅ ❌ 📝) instead of text buttons.
  - Apply CSS transitions when toggling or deleting tasks.
- **Task Manager Class:**
  - Convert the app into a **class-based MVC structure** (ask the AI tool to tell you what is meant by MVC structure).